

Temporal-Difference Learning

TD(0) · SARSA · Q-Learning · Maximization Bias

Deep Reinforcement Learning — Self-Study Slide Deck

Original worked examples authored for self-study — not a copy of any lecturer slide deck.

Expected Learning Outcomes

1. Why TD methods matter in RL
2. How TD connects to DP and Monte Carlo
3. The framework of TD methods — TD(0), TD target, TD error
4. SARSA vs Q-Learning — how they differ, how each works
5. Maximization bias — what it is, its impact, and the fix

Bottom line for the exam

You should be able to **trace an update by hand** for TD(0), SARSA, and Q-Learning given a small state diagram — not just recite the formulas.

Where TD Sits: Recap of DP and MC

Dynamic Programming

- Needs a full model $p(s', r \mid s, a)$
- Sweeps over *all* states
- Bootstraps: uses current V estimate of successor states

Monte Carlo

- No model needed — learns from sampled episodes
- Must wait until the *episode ends* to update
- Uses the actual return G_t , no bootstrapping

TD = the best of both

No model needed (like MC) **and** updates after a single step, using a bootstrapped estimate (like DP). This is why TD is the workhorse of practical RL.

1. Why TD Methods Matter

- **Model-free:** learns directly from raw experience, no transition probabilities needed.
- **Online:** updates *every step*, not just at the end of an episode — works for continuing (non-terminating) tasks too.
- **Bootstrapping:** uses its own current estimate of the next state's value as part of the update target — “an estimate improves an estimate.”
- **Low variance:** because the target only looks one step ahead, TD estimates are far less noisy than MC's full-return estimates.
- **Provably converges** to the correct value function under standard step-size conditions.

These five properties are exactly why almost every modern RL algorithm (Q-Learning, DQN, Actor-Critic, PPO) is built on a TD backbone.

2. DP vs MC vs TD — the Target Comparison

Method	Update target	Needs
DP	$\mathbb{E}[R + \gamma V(S')]$	full model
MC	G_t (actual full return)	complete episode
TD(0)	$R_{t+1} + \gamma V(S_{t+1})$	one step only

Read it this way

DP averages over *all possible* next states (using the model). MC uses the *one* return it actually observed, for the whole episode. TD does the MC thing (sample one transition) but the DP thing (bootstrap off $V(S_{t+1})$) for everything after it.

3. Framework — Building the TD(0) Update

Step 1. We want $V(S_t)$ to move toward a better estimate of the true return.

$$V(S_t) \leftarrow V(S_t) + \alpha [\text{target} - V(S_t)]$$

Step 2. MC's target is the full return G_t — but that needs the whole episode. Replace it with *one real reward* plus the *estimated* value of what comes next:

$$\text{target} \approx R_{t+1} + \gamma V(S_{t+1})$$

Step 3. Substitute back in:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD target}} - V(S_t) \right]$$

- $\alpha \in (0, 1]$ — step size (learning rate)
- $\gamma \in [0, 1]$ — discount factor

TD Target and TD Error

TD error δ_t

$$\delta_t = \underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD target}} - V(S_t)$$

- δ_t is the *surprise*: how wrong was $V(S_t)$, given what actually happened one step later?
- $\delta_t > 0$: things turned out *better* than expected — raise $V(S_t)$.
- $\delta_t < 0$: things turned out *worse* than expected — lower $V(S_t)$.
- The update is simply $V(S_t) \leftarrow V(S_t) + \alpha \delta_t$.

Exam one-liner

“TD error” and “TD target minus current estimate” are the same thing — if asked to define δ_t , write the boxed formula above.

TD(0) — Worked Example: a 4-cell Corridor

Setup: corridor $C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow \text{Terminal}$. Entering the terminal cell gives reward $+1$; every other move gives 0 . $\gamma = 1$, $\alpha = 0.5$. All $V = 0$ initially. Update states *in the order visited*, using the freshest value available at update time.

Episode 1 ($C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow T$):

$$V(C_1) \leftarrow 0 + 0.5[0 + 1.0 - 0] = 0$$

$$V(C_2) \leftarrow 0 + 0.5[0 + 1.0 - 0] = 0$$

$$V(C_3) \leftarrow 0 + 0.5[1 + 1.0 - 0] = \mathbf{0.5}$$

Episode 2 (same path):

$$V(C_1) \leftarrow 0 + 0.5[0 + 0 - 0] = 0$$

$$V(C_2) \leftarrow 0 + 0.5[0 + 0.5 - 0] = \mathbf{0.25}$$

$$V(C_3) \leftarrow 0.5 + 0.5[1 - 0.5] = \mathbf{0.75}$$

What to notice

The $+1$ reward leaks backward *one cell per episode*: C_3 learns it first, then C_2 “feels” it through C_3 , and eventually C_1 would too. This step-by-step backward leak is the signature behaviour of TD bootstrapping.

From Prediction to Control

- TD(0) estimates $V(s)$ — “how good is this *state*?”
- To actually choose actions, we need $Q(s, a)$ — “how good is this *action*, in this state?”
- **SARSA** and **Q-Learning** are the two classic TD *control* algorithms — they both learn $Q(s, a)$, using a TD-style update.
- They differ in exactly *one* place: what value they plug in for “what happens next.”

4. SARSA — On-Policy TD Control

The name

Uses the quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ — **S-A-R-S-A**.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- A_{t+1} is the action the agent *actually takes next*, chosen by its current policy (e.g. ϵ -greedy).
- You must pick A_{t+1} *before* you can perform the update.
- Because it plugs in the real, possibly-exploratory next action, SARSA learns the value of the policy it is *actually following* — it is **on-policy**.

SARSA — Algorithm

Pseudocode

```
Initialise  $Q(s,a) \leftarrow 0$  for all  $s,a$ 
For each episode:
   $S \leftarrow$  initial state
   $A \leftarrow$  choose from  $S$  using policy (e.g. eps-greedy on  $Q$ )
  Repeat until  $S$  is terminal:
    Take action  $A$ ; observe  $R, S'$ 
     $A' \leftarrow$  choose from  $S'$  using policy
     $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma Q(S',A') - Q(S,A)]$ 
     $S \leftarrow S'; A \leftarrow A'$ 
```

Key line

A' is chosen *before* the update line runs — SARSA always knows its own next move.

SARSA — Worked Example: a 2×2 Maze

Setup: states p, q, r, s . From p : action $exit \rightarrow +8$. From s : action $exit \rightarrow -60$. All other moves give 0. $\gamma = 0.8$, $\alpha = 1$ (clean numbers), all $Q = 0$ at start.

Episode 1: start at q . $A = go-p$ ($R = 0$) $\rightarrow p$;
policy then picks $A' = exit$.

$$Q(q, go-p) \leftarrow 0 + 1[0 + 0.8 \cdot 0 - 0] = 0$$

Next: $S = p$, $A = exit$, $R = 8$, $S' = T$.

$$Q(p, exit) \leftarrow 0 + 1[8 + 0 - 0] = 8$$

Episode 2: start at q again, same path.

$$Q(q, go-p) \leftarrow 0 + 1[0 + 0.8 \cdot \underbrace{8}_{Q(p, exit)} - 0] = 6.4$$

Value has propagated one step back,
exactly as with TD(0).

Key insight

SARSA corrects itself after *every single transition* — no need to wait for the episode to finish, unlike Monte Carlo.

5. Q-Learning — Off-Policy TD Control

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \underbrace{\max_{a'} Q(S_{t+1}, a')}_{\text{greedy over all actions}} - Q(S_t, A_t) \right]$$

- Instead of the *actual* next action, plug in the **best possible** action from S_{t+1} .
- No need to choose A_{t+1} first — update the instant S' is observed.
- Learns the value of the *optimal greedy* policy, even while the agent is still behaving randomly to explore — it is **off-policy**.
- Converges to q^* as long as every (s, a) pair is visited often enough.

Q-Learning — Algorithm

Pseudocode

```
Initialise  $Q(s,a) \leftarrow 0$  for all  $s,a$   
For each episode:  
   $S \leftarrow$  initial state  
  Repeat until  $S$  is terminal:  
     $A \leftarrow$  choose from  $S$  using policy (e.g. eps-greedy on  $Q$ )  
    Take action  $A$ ; observe  $R, S'$   
     $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_{a'} Q(S',a') - Q(S,A)]$   
     $S \leftarrow S'$ 
```

Key line

No A' appears anywhere — the update only ever needs S' .

Q-Learning — Worked Example: Falling Into the Pit

Same maze as before. Agent starts at s , takes action *exit*:

$$Q(s, \text{exit}) \leftarrow 0 + 1[-60 + 0.8 \cdot 0 - 0] = -\mathbf{60}$$

Next episode: agent at r , considers “go to s .” Q-Learning needs $\max_{a'} Q(s, a') = \max(Q(s, \text{exit})=-60, Q(s, \text{go-}r)=0) = 0$.

- So the -60 penalty does *not* propagate yet from this single visit.
- Only once $Q(s, \text{go-}r)$ itself turns negative (through more episodes) does the penalty reach further back.
- Eventually Q-Learning learns to route the agent away from s altogether, toward the $+8$ exit at p .

SARSA vs Q-Learning — the Key Difference

SARSA (on-policy)	Q-Learning (off-policy)
Target uses <i>actual</i> next action $Q(S', A')$	Target uses <i>max</i> over all next actions $\max_{a'} Q(S', a')$
Learns the value of the exploring policy — cautious near cliffs/pits	Learns the value of the optimal policy directly — often faster to a good policy
Needs A' chosen before updating	Updates the instant S' is observed

Exam trap

The two update equations have identical structure. The *only* line that differs when tracing one step of each is $Q(S', A')$ (SARSA) vs $\max_{a'} Q(S', a')$ (Q-Learning).

6. Maximization Bias — What It Is

The core inequality

$$\mathbb{E}[\max_a Q(s, a)] \geq \max_a \mathbb{E}[Q(s, a)]$$

- Early in training, every $Q(s, a)$ is a *noisy* estimate (some too high, some too low, purely from sampling luck).
- Taking a max over several noisy estimates tends to pick out one that got *lucky high* — not the action that is actually best.
- Q-Learning's update uses exactly this max every single step $\Rightarrow$ it systematically **overestimates** action values.
- **Impact:** the agent can become overconfident about mediocre actions, slows convergence, and in stochastic environments can make learning unstable.

Maximization Bias — Worked Example

Setup: from state A , action *right* goes straight to the terminal for a guaranteed reward 0. Action *left* goes to state B , which has **4** possible actions, each paying a reward drawn from $\mathcal{N}(-0.2, 1)$ before terminating. *True* best action from A is *right* (value $0 > -0.2$).

Suppose after a few sampled visits the 4 action estimates at B happen to be:

$$Q(B, a_1)=0.3, \quad Q(B, a_2)=-0.5, \quad Q(B, a_3)=-0.1, \quad Q(B, a_4)=-0.4$$

Q-Learning's target for $Q(A, \text{left})$ uses $\max_a Q(B, a) = \mathbf{0.3}$ — even though the *true* mean is -0.2 .

Result

The agent's estimate says “going left looks great (0.3)” when the truth is “going left is bad (-0.2).” It will keep choosing *left* far more often than it should, purely from sampling noise being run through a max.

Fixing It — Double Q-Learning

Idea: keep *two* independent estimators Q^A and Q^B . Use one to pick the best action, and the *other* to evaluate it — so the same noisy sample can't both choose *and* score the winner.

$$Q^A(S, A) \leftarrow Q^A(S, A) + \alpha \left[R + \gamma Q^B(S', \arg \max_{a'} Q^A(S', a')) - Q^A(S, A) \right]$$

(swap $A \leftrightarrow B$ on alternating updates)

- Splitting the same 4 samples from the last slide into two halves — say $Q^A(B, \cdot) = \{0.3, -0.1\}$ and $Q^B(B, \cdot) = \{-0.5, -0.4\}$ — Q^A picks a_1 (value 0.3) as the argmax, but Q^B *evaluates* a_1 using its own independent estimate, which is not inflated by the same noise.
- The resulting evaluation sits much closer to the true -0.2 than the raw single-estimator max of 0.3.

Cost

Twice the memory (two Q -tables) and slightly slower learning per update, in exchange for an unbiased estimate.

Summary Table — DP, MC, SARSA, Q-Learning

	DP	MC	SARSA	Q-Learning
Needs model?	Yes	No	No	No
Bootstraps?	Yes	No	Yes	Yes
Waits for episode end?	—	Yes	No	No
On/off policy?	—	either	On	Off
Target	$\mathbb{E}[R + \gamma V(S')]$	G_t	$R + \gamma Q(S', A')$	$R + \gamma \max_{a'} Q(S', a')$

Key Takeaways for the Exam

- TD = model-free (like MC) + bootstrapping (like DP) + online, single-step updates.
- TD(0) update: $V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$. The bracketed term is the **TD error** δ_t .
- SARSA and Q-Learning share the same update *shape*; the only difference is $Q(S', A')$ vs $\max_{a'} Q(S', a')$.
- SARSA = on-policy = cautious, learns what it actually does. Q-Learning = off-policy = learns the optimal policy regardless of how it explores.
- Maximization bias: $\mathbb{E}[\max Q] \geq \max \mathbb{E}[Q]$ — Q-Learning's max step systematically overestimates. Double Q-Learning fixes this with two decoupled estimators.

End of Deck

Concepts follow Sutton & Barto, *Reinforcement Learning: An Introduction*, 2nd ed., Chapter 6
— all worked-example numbers on these slides are original, authored for self-study.